"@Endpoint and @PayloadRoot turn a signed contract into working code."

MODULE 24

SOAP Web Service Endpoints and Request Mapping

Build, secure, and integrate contract-first SOAP web services using Spring Web Services (Spring WS) endpoints.

TEKsystems Global Services

WORKFORCE
DEVELOPMENT

WHY SOAP REMAINS IMPORTANT

SOAP continues to power mission-critical integrations where security, reliability, and standards matter most.

While REST has become popular, SOAP continues to be critical in many enterprise environments.

- **Enterprise-Grade Security —** built-in support for WS-Security standards provides message-level security, encryption, signatures, and authentication.
- **Formal Contract and Strong Typing —** WSDL and XML Schema enforce strict contracts, ensuring clarity, validation, and robust integration.
- **Reliability and Standards —** uses proven protocols (HTTP, SMTP, etc.) and standards supported by major vendors and organizations.
- **Enterprise and Legacy Integration —** ideal for integrating with legacy systems, mainframes, and heterogeneous platforms.
- **Governance and Compliance —** meets regulatory and compliance requirements in industries like banking, healthcare, and insurance.

SOAP IS STILL RELEVANT WHERE...

- Secure by Design
- Standards Based
- Reliable Messaging
- Compliance Ready
- Legacy System Integration
- Enterprise Adoption

KEY TAKEAWAY SOAP continues to power mission-critical integrations where security, reliability, and standards matter most.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design, develop, and secure SOAP web services using Spring WS.

- **Review SOAP Fundamentals —** recall SOAP architecture, messages, WSDL, and XML Schema concepts.
- **Understand Spring Web Services —** explain the purpose, architecture, and core components of Spring WS.
- **Apply Contract-First Development —** design SOAP services using WSDL and XML Schema before writing code.
- **Build SOAP Endpoints —** create and configure endpoints using @Endpoint and @PayloadRoot.
- **Process SOAP Requests —** implement the request/response processing workflow and message lifecycle.
- **Work with XML —** perform XML marshalling and unmarshalling of SOAP payloads.
- **Handle SOAP Faults —** structure and translate business errors into standards-compliant SOAP faults.
- **Implement WS-Security —** apply WS-Security concepts, including UsernameToken, to secure SOAP services.
- **Integrate Enterprise SOAP Services —** consume and expose SOAP services in enterprise environments.
- **Follow Enterprise Best Practices —** apply best practices for reliable, secure, maintainable SOAP services.

KEY TAKEAWAY These objectives build the skills to design, build, secure, and integrate production-grade SOAP endpoints with Spring WS.

SOAP ARCHITECTURE RECAP

A brief refresher -- Module 13 already covered SOAP envelope, headers, faults, and versioning in depth.

SOAP (Simple Object Access Protocol) is a protocol for exchanging structured information in a decentralized, distributed environment.

SOAP MESSAGE STRUCTURE

Part	Role
SOAP Envelope	The root element that defines the XML document as a SOAP message.
SOAP Header (Optional)	Carries metadata such as security, transactions, routing, and other context.
SOAP Body (Mandatory)	Carries the actual request or response data (the payload).
Transport Layer	Typically HTTP/HTTPS, SMTP, or JMS -- moves the SOAP message between client and server.

HOW IT WORKS

1. **Client Request**
2. **Wrap in Envelope**
3. **Send via Transport**
4. **Server Processes**
5. **Generate Response**
6. **Return to Client**

KEY TAKEAWAY SOAP's formal envelope, header, and body structure keeps it standards-based and enterprise-ready.

WSDL REVIEW (1 OF 2)

WSDL (Web Services Description Language) is an XML-based language that describes a SOAP service and how to access it.

PURPOSE OF WSDL

- Describes what the service does, how it can be accessed, where it is located, and what data formats it uses.

KEY COMPONENTS OF WSDL

Component	Defines
types	Reusable data types used by the service, defined using XML Schema (XSD).
message	The input and output messages for the operations.
portType	The interface -- the set of operations (methods) the service exposes.
binding	The protocol (SOAP) and data format (e.g. document/literal) used to communicate.
service	The actual endpoint URL where the service can be accessed.

KEY TAKEAWAY types -> message -> portType -> binding -> service: WSDL builds from reusable data types up to a live, addressable endpoint.

WSDL REVIEW (2 OF 2)

A sample WSDL document, how it is used end to end, and why it matters for contract-first integration.

Sample WSDL Snippet

```
<definitions name="CustomerService" targetNamespace="http://northstar.com/crm/customer"
             xmlns:tns="http://northstar.com/crm/customer" xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <!-- types --> <types> ... </types>
  <!-- messages --> <message name="GetCustomerRequest"> ... </message>
  <!-- portType --> <portType name="CustomerServicePort"> ... </portType>
  <!-- binding --> <binding name="CustomerBinding" type="tns:CustomerServicePort">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  </binding>
  <!-- service --> <service name="CustomerService">
    <port name="CustomerPort" binding="tns:CustomerBinding">
      <soap:address location="http://localhost:8080/ws"/>
    </port>
  </service>
</definitions>
```

HOW WSDL IS USED

- Provider Creates WSDL
- Client Reads WSDL
- Client Generates Stub
- Exchange SOAP Messages

WHY WSDL MATTERS

- Contract-First Dev
- Loose Coupling
- Interoperability
- Legacy Integration

KEY TAKEAWAY DefaultWsdl11Definition generates WSDL dynamically from your XSD, so it never drifts from the contract.

XML SCHEMA REVIEW (1 OF 2)

XML Schema (XSD) defines the structure, data types, and constraints for XML documents -- the foundation for strong typing in SOAP.

KEY CONCEPTS

Concept	Description
Elements	Define the structure and hierarchy of XML data.
Data Types	Define the type of data (string, int, boolean, date, etc.).
Constraints	Apply rules like minOccurs, maxOccurs, length, pattern, etc.
Complex Types	Define elements that contain other elements or attributes.
Attributes	Provide additional information about elements.

XML SCHEMA STRUCTURE

1. **xs:schema** — Root element
2. **xs:element** — Defines elements
3. **xs:complexType** — Complex structures
4. **xs:simpleType** — Custom simple types
5. **xs:attribute** — Defines attributes

KEY TAKEAWAY XSD is reused by WSDL to define message formats for SOAP operations -- it is the single source of truth for the contract.

XML SCHEMA REVIEW (2 OF 2)

A sample XSD, its benefits, and where XML Schema is used across SOAP web services.

Sample XML Schema (XSD) -- customer.xsd

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           xmlns:tns="http://northstar.com/crm/customer"
           targetNamespace="http://northstar.com/crm/customer" elementFormDefault="qualified">
  <xs:simpleType name="CustomerStatus">
    <xs:restriction base="xs:string">
      <xs:enumeration value="PROSPECT"/> <xs:enumeration value="ACTIVE"/>
      <xs:enumeration value="SUSPENDED"/> <xs:enumeration value="CLOSED"/>
    </xs:restriction>
  </xs:simpleType>
  <xs:complexType name="CustomerType">
    <xs:sequence>
      <xs:element name="customerId" type="xs:string"/>
      <xs:element name="status" type="tns:CustomerStatus"/>
    </xs:sequence>
  </xs:complexType>
</xs:schema>
```

BENEFITS OF XML SCHEMA

- Validates XML
- Enforces Data Types
- Improves Interoperability
- Reduces Integration Errors

COMMON USE CASES

- Request/Response Structures
- Data Validation
- JAXB Code Generation

KEY TAKEAWAY jaxb2-maven-plugin generates Java classes directly from this XSD -- the schema drives the code.

WHAT IS SPRING WEB SERVICES?

Spring WS helps you build robust, standards-compliant SOAP web services that interoperate with any platform or language.

Spring Web Services (Spring WS) is a lightweight, extensible framework for building SOAP-based web services and clients in the Java ecosystem. It simplifies contract-first SOAP development using XML, WSDL, and XSD.

KEY HIGHLIGHTS

Standards Based

Built on SOAP, WSDL, XML Schema, and other open standards.

Contract-First

Service contracts are defined using WSDL and XSD before implementation.

Lightweight & Flexible

Lightweight vs. full JAX-WS; integrates easily with the Spring ecosystem.

Enterprise Ready

Supports WS-Security, logging, validation, exception handling, monitoring.

COMMON USE CASES

- Enterprise System Integration
- B2B / Partner Integrations
- Legacy System Modernization
- Compliance-Driven Solutions
- Cross-Org Data Exchange

KEY TAKEAWAY Spring WS helps you build robust, standards-compliant SOAP web services that interoperate with any platform or language.

SPRING WS ARCHITECTURE

Spring Web Services provides a flexible, extensible, contract-first framework for building SOAP-based web services.

Component	Responsibility
Message Dispatcher	Receives incoming SOAP requests and dispatches them to the correct endpoint.
Endpoint Adapter	Bridges the transport layer and the endpoint by invoking the business method.
@Endpoint	POJO annotated with @Endpoint containing the business logic for the service.
Marshalling / Unmarshalling	Uses JAXB (or another marshaller) to convert between Java objects and XML.
Message Factory	Creates and processes SOAP messages and manages SOAP headers and payload.
Transport Layer	Responsible for delivering messages over HTTP, SMTP, JMS, etc.

SOAP REQUEST PROCESSING FLOW

1. **Client Sends**
2. **Transport Receives**
3. **Dispatcher Maps**
4. **Invokes Endpoint**
5. **Endpoint Runs**
6. **Marshals Reply**

KEY TAKEAWAY Cross-cutting support (security, validation, logging) wraps every step from Dispatcher through Message Factory.

CONTRACT-FIRST DEVELOPMENT (1 OF 2)

Contract-First Development means defining the service contract (WSDL and XML Schema) before writing any code.

The contract acts as an agreement between the service provider and consumers, ensuring a clear and stable interface. After gathering requirements, design the service interface first, agree on the contract, then implement -- this reduces rework and enables parallel development.

CONTRACT-FIRST DEVELOPMENT PROCESS

1. **Define Contract**
2. **Review & Approve**
3. **Generate Code**
4. **Implement Service**
5. **Develop Client**

The contract (WSDL + XSD) is the single source of truth and remains stable over time -- providers and consumers can then work independently against it.

KEY TAKEAWAY In Lab 24 the XSD is authored first (Step 2); the WSDL, JAXB types, and endpoint all follow from it -- never the reverse.

CONTRACT-FIRST DEVELOPMENT (2 OF 2)

Key artifacts, benefits, and best practices for contract-first SOAP development.

KEY ARTIFACTS

- **XSD (XML Schema) —** defines the structure, data types, and constraints of request/response messages.
- **WSDL —** describes the service, operations, message formats, and how to access it.
- **Service Contract —** the agreement between provider and consumer that should not change frequently.
- **Generated Artifacts —** server stubs, client proxies, and data bindings (via wsimport, xjc, or JAXB).

BENEFITS

Clear Agreement · **Parallel Development** · **Reduced Rework** · **Consistency** · **Maintainability**

BEST PRACTICES

- Invest time in designing a robust contract with clear documentation.
- Version your contracts (e.g., v1, v2) and maintain backward compatibility.
- Automate contract validation in the build pipeline.

KEY TAKEAWAY A well-designed, versioned contract is the single source of truth for every consumer.

TRY IT YOURSELF — EXERCISE 1: CONTRACT-FIRST RECALL

Reinforces: the XSD -> JAXB -> WSDL contract-first order and the Lab 13 link you just covered.

STEP / WHAT TO DO

Step	What To Do
Goal	Explain why the partner XSD -- not Java classes -- owns the SOAP contract.
File	notes/contract-first.md (in examples/module-24-exercises/).
Order	Align your explanation with the XSD -> JAXB -> WSDL order: XSD is the source of truth.
Lab 13 link	Note that Lab 24 implements Lab 13's customer operations over Spring-WS.
Deliverable	Contract-first rule and Lab 13 link documented; state you will not author the full XSD yet.
Reference	exercises/exercise-01-contract-first-recall.md

Contract-First Flow (restate this in your own words in notes/contract-first.md)

```
customer.xsd  (hand-authored -- the single source of truth)
   |  mvn generate-sources (jaxb2-maven-plugin)          v  DefaultWsdl11Definition
   v                                                     v
Generated JAXB types (derived, never hand-edited)   Dynamic WSDL /ws/customer.wsdl (never hand-written)
```

TRY IT NOW Complete Exercise 1 before continuing -- see exercises/exercise-01-contract-first-recall.md.

ENDPOINT ARCHITECTURE

In Spring Web Services, an endpoint is the server-side component that receives, processes, and responds to SOAP requests.

WHAT IS AN ENDPOINT?

- **Business Function —** an endpoint represents a specific business function exposed as a SOAP web service operation.
- **Annotation-Mapped —** handles incoming SOAP requests and maps them to Java methods using annotations.
- **Runtime-Managed —** operates within the Spring WS infrastructure for message handling, marshalling, and security.
- **Focused & Reusable —** kept focused on request handling and easy to maintain.

SPRING WS ENDPOINT RUNTIME

1. **Dispatcher**
2. **Endpoint**
3. **Biz Logic**
4. **Marshal**
5. **Msg Factory**

BENEFITS OF THIS ARCHITECTURE

Loose Coupling | **Scalability** | **Maintainability** | **Standards Compliant**

KEY TAKEAWAY Endpoints are decoupled from clients and from the transport layer -- add or extend them independently as the contract grows.

@ENDPOINT ANNOTATION (1 OF 2)

The @Endpoint annotation marks a class as a SOAP endpoint -- Spring WS detects it and registers it as a web service endpoint.

WHAT IS @ENDPOINT?

- **Class-level annotation —** declares that a class contains methods that handle incoming SOAP requests.
- **Auto-registered —** allows Spring to scan and register the endpoint automatically.
- **Method mapping —** methods within the class are mapped to SOAP operations using @PayloadRoot.
- **Spring-native —** supports dependency injection and other Spring features.

HOW @ENDPOINT WORKS

1. **Spring Scans**
2. **Creates Bean**
3. **Registers w/ Dispatcher**
4. **Routes SOAP Requests**
5. **Method Handles**

KEY TAKEAWAY @Endpoint promotes clean separation of concerns -- delegate business logic to service classes and keep endpoints stateless.

@ENDPOINT ANNOTATION (2 OF 2)

A working @Endpoint example, its annotation details, and best practices for endpoint classes.

CustomerEndpoint.java

```java
@Endpoint
public class CustomerEndpoint {
    private static final String NS = "http://northstar.com/crm/customer";

    private final CustomerService customerService;

    @PayloadRoot(namespace = NS, localPart = "getCustomerRequest")
    @ResponsePayload
    public GetCustomerResponse getCustomer(
            @RequestPayload GetCustomerRequest request) {
        // delegate to the same service REST uses
        Customer c = customerService.get(request.getCustomerId());
        return CustomerSoapMapper.toSoap(c);
    }
}
```

ANNOTATION DETAILS

Field	Value
Package	org.springframework.ws.server.endpoint.annotation
Target	TYPE (Class)
Retention	RUNTIME
Purpose	Marks a class as a SOAP endpoint

BEST PRACTICES

- Use meaningful class names related to the domain.
- Delegate business logic to service classes -- never re-implement rules in the endpoint.
- Keep endpoints stateless whenever possible.

KEY TAKEAWAY CustomerEndpoint stays a thin protocol adapter -- it delegates so SOAP and REST never fork business rules.

REQUEST MAPPING WITH @PAYLOADROOT

@PayloadRoot maps an incoming SOAP request to a specific endpoint method based on the XML payload's namespace and local name.

Attribute	Meaning
namespace	Namespace URI of the root element in the request payload.
localPart	Name of the root element to be mapped (e.g. getCustomerRequest).

Example

```
@PayloadRoot(namespace = NS, localPart = "getCustomerRequest")
@ResponsePayload
public GetCustomerResponse getCustomer(
    @RequestPayload GetCustomerRequest request) { ... }
```

HOW REQUEST MAPPING WORKS

1. **SOAP Request**
2. **Message Dispatcher**
3. **@PayloadRoot Match**
4. **Handler Method**
5. **SOAP Response**

Simple & Declarative | **Supports Contract-First** | **Works with WSDL/XSD** | **Keeps Methods Focused**

KEY TAKEAWAY Use constants for the namespace URI -- four @PayloadRoot methods, one well-named method per operation.

TRY IT YOURSELF — EXERCISE 2: SOAP OPERATION MAP

Reinforces: mapping SOAP operations to shared CustomerService methods via @PayloadRoot you just saw.

STEP / WHAT TO DO

Step	What To Do
Goal	Map four customer SOAP operations to shared CustomerService methods.
File	notes/soap-ops.md (in examples/module-24-exercises/).
Map	CreateCustomer -> create; GetCustomer -> get by id; UpdateCustomerStatus -> status transition; ListCustomers -> list/filter.
Shared service rule	Write that REST and SOAP must share CustomerService so business rules never fork.
Deliverable	Operation map + shared-service rule; list fixtures CUS-1001, CUS-1002, CUS-9999, correlation lab24-001.
Reference	exercises/exercise-02-operation-map.md

The Map You Are Writing (localPart -> CustomerService method)

```
@PayloadRoot(namespace = NS, localPart = "createCustomerRequest")       -> customerService.create(...)
@PayloadRoot(namespace = NS, localPart = "getCustomerRequest")          -> customerService.get(...)
@PayloadRoot(namespace = NS, localPart = "updateCustomerStatusRequest") -> customerService.updateStatus(...)
@PayloadRoot(namespace = NS, localPart = "listCustomersRequest")        -> customerService.list(...)
```

TRY IT NOW Complete Exercise 2 before continuing -- see exercises/exercise-02-operation-map.md.

TRY IT YOURSELF — EXERCISE 3: PAYLOADROOT SKELETON (STARTER)

Reinforces: @Endpoint / @PayloadRoot delegation you just saw -- a fill-in-the-blank starter, not built from scratch.

notes/CustomerEndpointSketch.java (STARTER -- replace every blank)

```
@_____
public class CustomerEndpoint {
    private final CustomerService service;

    public CustomerEndpoint(CustomerService service) {
        this.service = service;
    }

    @PayloadRoot(namespace = "http://northstar.example/customer",
                 localPart = "_____")
    @ResponsePayload
    public GetCustomerResponse get(
            @RequestPayload GetCustomerRequest request) {
        // TODO: call service.get(id) then map to JAXB response
        var customer = service._____(request.getCustomerId());
        return CustomerSoapMapper.toGetResponse(customer);
    }
}
```

STEP / WHAT TO DO

Step	What To Do
Goal	Complete a pseudocode endpoint skeleton that delegates to CustomerService.
Starter	TODO / fill-in-the-blank; replace every _____ and // TODO -- blanks do not compile.
Hints	Class annotation @Endpoint; localPart GetCustomerRequest; method service.get(...).
Self-check	No business rules live inside the endpoint beyond mapping/delegation.
Deliverable	All three blanks filled; delegation rule clear.
Reference	exercises/exercise-03-payloadroot-skeleton.md

TRY IT NOW Complete Exercise 3 (fill in every blank) before continuing -- see exercises/exercise-03-payloadroot-skeleton.md.

REQUEST PROCESSING WORKFLOW

Spring WS processes a SOAP request through well-defined steps, from the transport layer to the endpoint method and back.

1 Transport
2 Receiver
3 Dispatcher
4 Endpoint Map
5 Unmarshal
6 Invoke
7 Marshal
8 Send

KEY COMPONENTS INVOLVED

- **MessageDispatcher —** central component that routes messages to the correct endpoint.
- **@PayloadRoot —** maps the XML root element (namespace + localPart) to the handler method.
- **Interceptors (Optional) —** used for logging, security, and validation at multiple points.

Step	What Happens
1-3. Receive & Dispatch	SOAP request arrives; MessageReceiver builds a MessageContext; MessageDispatcher reads the payload.
4-6. Map, Unmarshal & Invoke	@PayloadRoot matches; payload unmarshalled to a Java object; handler method invoked.
7-8. Marshal & Send	Return value marshalled to XML; SOAP response sent back to the client.

KEY TAKEAWAY A clear, predictable request flow with loose coupling between client and service -- and easy to extend with interceptors.

SOAP REQUEST LIFECYCLE

The journey of a request from the client to the Spring WS endpoint, until it is ready for the handler method.

1. **Transport** — HTTP(S) arrives
2. **Receiver** — Creates context
3. **Dispatcher** — Mapping rules
4. **Endpoint Map** — Via @PayloadRoot
5. **Interceptors** — Security, logging
6. **Ready to Handle** — Payload unmarshalled

WHAT HAPPENS IN THIS LIFECYCLE?

- The HTTP request is accepted and the SOAP envelope is parsed.
- The correct endpoint is resolved using @PayloadRoot mapping information.
- Cross-cutting concerns (security, logging, validation) are applied via interceptors.
- The payload is converted from XML to Java objects before the handler runs.

KEY TAKEAWAY These steps occur for every incoming SOAP request before the business logic is executed.

SOAP RESPONSE LIFECYCLE

The journey of a response from the endpoint method back to the client, until it is received and processed.

1. **Endpoint Done** — Logic completes
2. **Marshalled** — Java object -> XML
3. **Message Built** — SOAP Body assembled
4. **Interceptors Run** — Headers, auditing
5. **Message Sent** — Sent via HTTP(S)
6. **Client Receives** — Client parses XML

WHAT IF SOMETHING GOES WRONG?

If an exception is thrown in the endpoint method, an appropriate SOAP Fault is created during this lifecycle and returned to the client instead of a normal response -- see the next section.

Consistent Handling | **Supports Cross-Cutting Concerns** | **Easier Troubleshooting**

KEY TAKEAWAY The response lifecycle mirrors the request lifecycle -- and it's where a thrown exception becomes a SOAP Fault.

XML MARSHALLING

XML Marshalling is the process of converting Java objects into XML so they can be included in a SOAP message.

1. **Java Object (POJO)**
2. **Marshaller**
3. **XML Output**
4. **SOAP Message**

Java Object

```
Customer customer = new Customer();
customer.setId("CUS-1001");
customer.setName("Amina Khan");
customer.setEmail("amina@northstar.com");
```

Marshalled XML (Output)

```
<customer>
  <id>CUS-1001</id>
  <name>Amina Khan</name>
  <email>amina@northstar.com</email>
</customer>
```

KEY POINTS

- Uses a Marshaller (typically JAXB / Jakarta XML Binding) to convert Java objects to XML.
- Follows the structure defined in the XSD or JAXB annotations -- always validate against the schema first.

KEY TAKEAWAY Marshalling ensures your Java data is correctly transformed into XML that follows the SOAP contract.

XML UNMARSHALLING

XML Unmarshalling converts XML from a SOAP request into Java objects so the application can process the data.

1 **SOAP Request (XML)**

2 **Unmarshaller**

3 **Java Object (POJO)**

4 **Business Processing**

SOAP Request XML

```
<getCustomerRequest>
  <customerId>CUS-1001</customerId>
</getCustomerRequest>
```

Java Object (Output)

```
GetCustomerRequest request =
    new GetCustomerRequest();
request.setCustomerId("CUS-1001");
// passed to the @RequestPayload parameter
```

KEY POINTS

- Extracts data from the SOAP request and validates the XML structure using XSD (if configured).
- Populates strongly-typed Java objects (POJOs) for business processing -- no manual XML parsing.

KEY TAKEAWAY Unmarshalling transforms XML into usable Java objects for clean, type-safe business logic.

SOAP FAULT STRUCTURE AND ERROR HANDLING

When an endpoint throws an exception, Spring WS converts it into a structured SOAP Fault instead of a raw stack trace.

ANATOMY OF A SOAP FAULT

- **faultcode —** a qualified code such as Client or Server, identifying who is at fault.
- **faultstring —** a short, human-readable description of the error (never a stack trace).
- **faultactor —** identifies which node in the message path caused the fault (optional).
- **detail —** optional structured, application-specific error information.

Fault Code	Meaning
Client	Caller sent a bad/invalid request (e.g. not-found, duplicate).
Server	Something failed on the server side; generic message, no internals.
MustUnderstand	A required SOAP header element was not understood.
VersionMismatch	The SOAP envelope version is not supported.

SoapFaultMappingExceptionResolver

```
@Bean
SoapFaultMappingExceptionResolver exceptionResolver() {
  var resolver = new SoapFaultMappingExceptionResolver();
  var mappings = new Properties();
  mappings.setProperty(
    "com.northstar.crm.exception.CustomerNotFoundException",
    "CLIENT,Customer not found");
  resolver.setExceptionMappings(mappings);
  return resolver;
}
```

Fault Response (CUS-9999, not found)

```
<SOAP-ENV:Fault>
  <faultcode>SOAP-ENV:Client</faultcode>
  <faultstring>Customer not found</faultstring>
</SOAP-ENV:Fault>
```

Never let faultstring or detail leak stack traces or secrets.

KEY TAKEAWAY Map business exceptions to CLIENT faults with clear messages; default everything else to SERVER -- no stack traces, ever.

TRY IT YOURSELF — EXERCISE 4: SOAP FAULT VS. REST ERROR

Reinforces: mapping business exceptions to SOAP faults without leaking stacks you just covered.

STEP / WHAT TO DO

Step	What To Do
Goal	Document how business exceptions become SOAP faults without leaking stacks.
File	notes/fault-vs-rest.md (in examples/module-24-exercises/).
Contrast table	Columns Concern | SOAP | REST; rows: not-found, validation, missing UsernameToken.
Answer sketch	Not-found -> SOAP fault vs HTTP 404 JSON; missing token -> security fault vs 401 on REST.
Deliverable	Contrast table + no-stack-trace rule; note Lab 16 exceptions feed Lab 24 fault mapping.
Reference	exercises/exercise-04-fault-vs-rest.md

Same Failure, Two Protocols (what your contrast table is documenting)

```
SOAP:  <SOAP-ENV:Fault><faultcode>SOAP-ENV:Client</faultcode><faultstring>Customer not found</faultstring></SOAP-ENV:Fault>
REST:  HTTP 404 { "error": "NOT_FOUND", "message": "Customer not found" }
Rule:  faultstring/detail and REST error bodies never include stack traces, class names, or secrets.
```

TRY IT NOW Complete Exercise 4 before continuing -- see exercises/exercise-04-fault-vs-rest.md.

SOAP SECURITY OVERVIEW

SOAP provides several standards and mechanisms (WS-Security) to ensure confidentiality, integrity, authentication, and non-repudiation.

GOALS OF SOAP SECURITY

Goal	What It Ensures
Confidentiality	Message content is not accessible to unauthorized parties.
Integrity	Message content is not altered in transit.
Authentication	Verifies the identity of the sender and/or receiver.
Non-Repudiation	Provides cryptographic proof of origin and prevents denial of actions.

WS-SECURITY STANDARD

WS-Security is an OASIS standard that extends SOAP to add security tokens and related processing to SOAP messages -- independent of the transport layer.

COMMON SECURITY TOKENS

UsernameToken

SAML Token

X.509 Certificate

BinarySecurityToken

Always use HTTPS for transport security in addition to WS-Security -- message-level security and transport security solve different problems.

KEY TAKEAWAY SOAP security combines HTTPS transport with message-level WS-Security so identity survives intermediaries.

WS-SECURITY CONCEPTS

WS-Security extends SOAP messages by adding security information in the SOAP Header -- most commonly a UsernameToken.

wsse:Security Header (UsernameToken)

```
<soapenv:Header>
  <wsse:Security xmlns:wsse="...oasis-200401-wss-wssecurity-secext-1.0.xsd">
    <wsse:UsernameToken>
      <wsse:Username>crm-partner</wsse:Username>
      <wsse:Password Type="...PasswordText">
        lab24-shared-secret
      </wsse:Password>
    </wsse:UsernameToken>
  </wsse:Security>
</soapenv:Header>
```

CORE WS-SECURITY CONCEPTS

- **Security Tokens —** carry authentication info about the sender (UsernameToken, SAML, X.509).
- **Digital Signature —** ensures integrity and authenticity using XML Signature.
- **Encryption —** protects message content using XML Encryption.
- **Timestamp —** prevents replay attacks by adding creation time and expiration.

HOW A WSS4J USERNAMETOKEN INTERCEPTOR WORKS

1. **SOAP Msg + Header**
2. **Wss4j Interceptor**
3. **Validates Token**
4. **Reject or Forward**
5. **Endpoint Runs**

KEY TAKEAWAY Plaintext PasswordText UsernameToken is lab-only -- production needs TLS plus PasswordDigest (or better) and rotated secrets.

TRY IT YOURSELF — EXERCISE 5: USERNAMETOKEN PLAN

Reinforces: WSS4J UsernameToken validation you just saw -- planning the evidence, not implementing JWT.

STEP / WHAT TO DO

Step	What To Do
Goal	Outline UsernameToken evidence without implementing JWT.
File	notes/usernametoken-plan.md (in examples/module-24-exercises/).
Happy path	Secured GetCustomer for CUS-1001 (crm-partner / lab24-shared-secret) succeeds.
Failure path	Missing/invalid token produces a distinct security fault from a not-found fault.
Deliverable	Secret hygiene rule (never real prod passwords) stated; Bearer JWT explicitly deferred to Lab 28.
Reference	exercises/exercise-05-usernametoken-plan.md

The Two Scenarios You Are Planning For

```
requests/get-customer-secured.xml   -> crm-partner / lab24-shared-secret UsernameToken -> Wss4j validates -> CustomerEndpoint runs
requests/get-customer.xml           -> no Security header                              -> Wss4j rejects  -> SOAP security fault (never reaches endpoint)
```

TRY IT NOW Complete Exercise 5 before continuing -- see exercises/exercise-05-usernametoken-plan.md.

INTEGRATING SOAP SERVICES (1 OF 2)

Integrating SOAP services means consuming external SOAP web services in your application using standard SOAP messages.

WAYS TO INTEGRATE SOAP SERVICES

- **1. WSDL-First Approach —** use the service's published WSDL to generate client stubs and strongly typed Java classes.
- **2. Contract-First Approach —** define your own contract (WSDL/XSD) first, then generate client and server artifacts.
- **3. Dynamic / Template Approach —** use WebServiceTemplate for dynamic requests without generating stubs.

INTEGRATION FLOW (SOAP CLIENT)

1. Discover Service
2. Generate Client
3. Configure Client
4. Send Request
5. Receive Response
6. Process Result

KEY TAKEAWAY Integrating SOAP services ensures seamless, standards-compliant communication with external systems.

INTEGRATING SOAP SERVICES (2 OF 2)

A sample Spring WS client, best practices, and how to handle SOAP faults from a downstream partner service.

Sample SOAP Client Code (Spring WS)

```
WebServiceTemplate template = new WebServiceTemplate();
template.setDefaultUri("https://partner.example.com/soap/CustomerService");

QName qname = new QName("http://northstar.com/crm/customer", "getCustomerRequest");
GetCustomerRequest request = new GetCustomerRequest();
request.setCustomerId("CUS-1001");
GetCustomerResponse response = (GetCustomerResponse) template.marshalSendAndReceive(qname, request);
```

BEST PRACTICES

- Always use HTTPS; validate XML with XSD before sending.
- Use WS-Security for authentication and integrity.
- Handle SOAP Faults and timeouts gracefully; cache WSDL locally to reduce latency.

HANDLING FAULTS FROM A PARTNER

- Always catch SoapFaultClientException around marshalSendAndReceive calls.
- Log the fault code, message, and correlation id -- never assume the call succeeded.
- Watch for WSDL changes, version drift, and large-payload latency in production.

KEY TAKEAWAY A resilient SOAP client treats faults, timeouts, and WSDL drift as first-class failure modes, not edge cases.

ENTERPRISE SOAP BEST PRACTICES (1 OF 2)

Following best practices ensures SOAP web services are secure, reliable, maintainable, and well-integrated.

#	Practice	What It Means
1	Use Contract-First Approach	Define WSDL and XSD first to establish a clear, stable contract.
2	Follow Naming Conventions	Use consistent, meaningful names for operations, messages, and types.
3	Keep Contracts Simple & Versioned	Avoid over-complicated schemas; version your services (e.g. /v1, /v2).
4	Validate with XSD	Always validate XML requests and responses against the schema.
5	Use WS-Security Standards	Implement authentication, integrity, and encryption using WS-Security.

KEY TAKEAWAY Contract discipline and schema validation are the foundation everything else -- security, reliability, observability -- builds on.

ENTERPRISE SOAP BEST PRACTICES (2 OF 2)

Apply these practices consistently to deliver reliable, maintainable, well-observed enterprise SOAP services.

#	Practice	What It Means
6	Enforce HTTPS	Always use SSL/TLS for transport security, in addition to WS-Security.
7	Handle Faults Gracefully	Use SOAP Faults to return meaningful, non-leaking error information.
8	Use Idempotent Operations	Design operations to be idempotent so retries are safe.
9	Centralize Logging & Correlation	Log requests, responses, and faults; include correlation IDs for tracing.
10	Monitor, Alert & Document	Track service health, latency, and errors; keep contracts documented.

KEY TAKEAWAY Well-designed, secure, and observable SOAP services ensure long-term success in enterprise integration.

TRY IT YOURSELF — EXERCISE 6: LAB 24 READINESS CHECKLIST

Capstone check -- confirm the Lab 23 REST baseline is understood before you open the Spring-WS lab.

STEP / WHAT TO DO

Step	What To Do
Goal	Verify the Lab 23 REST baseline is understood before starting the Spring-WS lab.
File	notes/lab24-readiness.md (in examples/module-24-exercises/).
Dependency	Note that Lab 23's Boot app + CustomerService is required before Lab 24 begins.
Path	Write the copy target: examples/lab24-crm.
Evidence correlation	Record lab24-001 for SOAP evidence (REST may still use lab-request-001).
Deliverable	Readiness note ties SOAP to the Boot baseline; list Kafka and React as Week 4, not Lab 24.

Readiness Checklist (notes/lab24-readiness.md)

```
[ ] Lab 23 Boot app + CustomerService exist and run          [ ] Copy target confirmed: examples/lab24-crm
[ ] Correlation ids noted: lab24-001 (SOAP), lab-request-001 (REST)    [ ] Kafka + React deferred to Week 4
```

TRY IT NOW Complete Exercise 6 before continuing -- see exercises/exercise-06-lab24-readiness.md.

LAB OVERVIEW -- SOAP WEB SERVICE ENDPOINTS

Lab 24: SOAP Web Service Endpoints — Northstar CRM Spring-WS

BUSINESS SCENARIO

- A regional billing partner only integrates via SOAP/XML and must create, get, update status, and list customers using the Lab 13 contract.
- Ship Spring-WS beside the Lab 23 REST API: live WSDL from XSD, four operations, CLIENT faults for not-found/duplicate, UsernameToken required.
- Protocol may differ; business rules must not -- both protocols delegate to the same CustomerService.

LEARNING OBJECTIVES

- Explain contract-first SOAP and why the XSD -- not Java -- is the source of truth.
- Author an XSD and let Spring-WS generate WSDL dynamically from it.
- Implement @Endpoint with @PayloadRoot / @RequestPayload / @ResponsePayload.
- Translate BusinessException subtypes into structured SOAP faults.
- Configure a minimal WS-Security UsernameToken interceptor.

WHAT YOU BUILD

- Copy lab23-crm to lab24-crm; add Spring-WS + jaxb2 plugin; author customer.xsd; configure MessageDispatcherServlet + DefaultWsdl11Definition; implement CustomerEndpoint with four @PayloadRoot operations; add SOAP fault mapping and a UsernameToken interceptor; verify with curl + MockWebServiceClient; evidence with correlation lab24-001.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE), CUS-1002 (Ravi Singh, PROSPECT), and CUS-9999 (not-found) anchor every example.

LAB TASKS AND DELIVERABLES

Northstar CRM Spring-WS -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch Lab 23 and add Spring-WS + jaxb2-maven-plugin dependencies.
2	Author customer.xsd and generate JAXB request/response classes.
3	Configure MessageDispatcherServlet and a live /ws/customer.wsdl.
4	Map JAXB types and implement CustomerEndpoint's four operations.
5	Share exceptions and map SOAP faults (CLIENT for not-found/duplicate).
6	Add a Wss4jSecurityInterceptor for UsernameToken (WS-Security).
7	Prove REST and SOAP share one CustomerService (same data, both protocols).
8	Automate with MockWebServiceClient tests + a partner evidence pack.

EXPECTED DELIVERABLES

- CustomerEndpoint with four SOAP operations + customer.xsd / live WSDL.
- JAXB generation + CustomerSoapMapper; SOAP fault mapping.
- Working UsernameToken interceptor (lab secret only).
- requests/ sample XML + fault cases; CustomerEndpointTest green twice.
- Evidence that REST and SOAP share CustomerService; README runbook.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security 10 · Docs 10

KEY TAKEAWAY Duplicating business rules or skipping WS-Security both lose core marks -- committing real secrets is an honor violation.

MODULE 24 SUMMARY

SOAP Web Service Endpoints and Request Mapping -- what we covered and how it fits together.

KEY CONCEPTS COVERED

Contract-First & Recap

SOAP architecture, WSDL, and XML Schema as the source of truth.

Endpoints & Mapping

@Endpoint and @PayloadRoot route SOAP requests to Java methods.

Lifecycle & Marshalling

Request/response lifecycle and XML marshalling/unmarshalling.

SOAP Faults

Structured faultcode/faultstring errors instead of stack traces.

WS-Security

UsernameToken and message-level security concepts.

Enterprise Best Practices

Contracts, security, reliability, and observability for SOAP.

MODULE FLOW RECAP

1. **Contract (WSDL/XSD)**
2. **@Endpoint/@PayloadRoot**
3. **Marshalling**
4. **Faults**
5. **WS-Security**

KEY TAKEAWAY You can now design, build, secure, and integrate production-ready SOAP endpoints with Spring WS -- contract-first, end to end.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of SOAP fundamentals, WSDL, and Spring WS endpoint annotations.

1. What is the role of WSDL in a SOAP web service?

- **A.** To describe the service implementation
- **B. To define the service contract, operations, and messages**
- **C.** To handle service security
- **D.** To store service logs

2. Which annotation is used to define a SOAP endpoint in Spring WS?

- **A.** @Service
- **B. @Endpoint**
- **C.** @WebService
- **D.** @Component

3. Which annotation maps an incoming request to a specific XML namespace and local part?

- **A.** @RequestMapping
- **B. @PayloadRoot**
- **C.** @Namespace
- **D.** @XmlRootElement

4. What is the purpose of XML Marshalling in Spring WS?

- **A.** Convert XML to Java objects
- **B. Convert Java objects to XML**
- **C.** Validate XML against XSD
- **D.** Encrypt XML messages

KEY TAKEAWAY WSDL, @Endpoint, and @PayloadRoot are the foundation of every Spring WS contract-first service.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on unmarshalling, WS-Security, contract-first design, and integration.

5. Which of the following is TRUE about XML Unmarshalling?

- **A.** It converts Java objects to XML
- **B. It converts XML into Java objects**
- **C.** It is used only for request messages
- **D.** It is not required in Spring WS

6. WS-Security primarily provides which capabilities?

- **A.** Compression of SOAP messages
- **B. Authentication, integrity, and confidentiality**
- **C.** Message routing and queuing
- **D.** Database encryption

7. Which approach is recommended for enterprise SOAP service development?

- **A.** Code-First approach
- **B. Contract-First approach**
- **C.** Database-First approach
- **D.** Manual-First approach

8. What is a key benefit of integrating SOAP services with WebServiceTemplate?

- **A.** Faster UI rendering
- **B. Loose coupling and reuse across systems**
- **C.** Elimination of databases
- **D.** Replacing REST APIs

KEY TAKEAWAY Contract-first design and WS-Security are what make a SOAP integration enterprise-ready.

KNOWLEDGE CHECK (3 OF 3)

Four scenario questions on SOAP faults, UsernameToken security, and shared business rules.

9. A CustomerNotFoundException is thrown inside CustomerEndpoint. What should the client receive?

- **A.** An HTTP 500 with a full Java stack trace
- **B. A SOAP Fault with faultcode Client and a clear faultstring**
- **C.** A silent empty response
- **D.** The connection is simply closed

10. True or False: A request without a valid UsernameToken should be rejected before business logic runs.

- **A. True — enforced before the endpoint method runs**
- **B.** False

11. Why must CustomerEndpoint delegate to CustomerService instead of re-implementing business rules?

- **A.** SOAP endpoints cannot call service classes
- **B. So REST and SOAP never diverge on business rules**
- **C.** It makes the WSDL generate faster
- **D.** JAXB requires it

12. True or False: HTTPS alone is a complete substitute for WS-Security UsernameToken.

- **A.** True
- **B. False — transport security and message-level security solve different problems**

KEY TAKEAWAY SOAP faults, WS-Security, and a single shared service are the three pillars of a trustworthy enterprise SOAP endpoint.

*"Contract-first, fault-aware, and secured --
SOAP done the enterprise way."*

MODULE 24 COMPLETE

SOAP Web Service Endpoints and Request Mapping

You can now design, build, secure, and integrate contract-first SOAP endpoints with Spring WS -- @Endpoint, @PayloadRoot, marshalling, faults, and WS-Security.